SICDT 4

Apresentação

Este documento apresenta uma proposta de arquitetura para o SICDT que dispõe na saída da aplicação atual do SIFEC que, ao mesmo tempo, torna o sistema mais independente e com uma nova interface de usuário utilizando tecnologias e frameworks modernos que são estabelecidos no mercado.

Problemas da atual arquitetura

A atual arquitetura e infraestrutura do SICDT enfrentam problemas que atingem o seu desenvolvimento e limitam uma experiência de usuário a um modelo não intuitivo e não prático de usabilidade. A interface é desenvolvida em fluxo gerenciado pelo Backbone com a manipulação de objetos DOM via jQuery e com a integração com o seu backend em antigo Java 2EE.

A infraestrutura do SICDT atual é bastante engessada, pouco prática em efetuar novas atualizações em produção mesmo com releases menores. É um processo lento e limitado que não consegue prover um fluxo rápido, independente se está desenvolvendo nele ou levando o artefato final à produção.

- Deploy de acoplamento lento: o ciclo completo de compilação de um artefato .ear envolve várias etapas e precisa necessariamente compilar toda a aplicação.
- Uso de ferramentas intermediárias para deploy: ter um CRM ou sistemas similares que gerem a compilação desses artefatos torna o processo de pipeline de entrega com uma dependência externa frágil e fora do controle direto do time.

- Processos complementares de qualidade são ausentes ou ineficientes: etapas como teste, análise estática e cobertura de código podem não ser implementadas ou são inexistentes, deixando o código não sendo validado de uma forma ampla durante o build.

Os problemas em adotar ou continuar usando um framework antigo como o Backbone são:

- Ausência de reatividade declarativa: exigindo uma atualização manual da UI a cada mudança de estado da página, o que consome mais recursos, tempo e deixa a lógica de processamento mais engessada.
- Sem sistemas de componentes: deixando a lógica de componentização sendo por outros frameworks ou até mesmo delegada para uma implementação mais crua em JS.
- Ecossistema morto: por ser um framework lançado em 2010 e, também nunca ter possuído uma popularidade expressiva, essa framework se encontra morto, sem evolução, sem atualizações e sem novas correções de segurança.
- Testabilidade baixa: a arquitetura orientada a eventos do Backbone torna testes unitários e de integração significativamente mais complexos do que em frameworks modernos.

Embora o framework javascript jQuery foi bastante útil e popular, ele se torna hoje um framework dispensável e ruim se comparado com advento das tecnologias atuais manipulação de DOM que são gerenciadas nativamente pelo JavaScript atual ES14+ (especialmente ES2025).

- Estado imprevisível: o DOM diretamente significa que o estado real da aplicação e o estado visual podem facilmente se dessincronizar, gerando bugs difíceis de rastrear.
- Acoplamento forte: lógica de negócio, manipulação visual e chamadas de rede frequentemente se misturam nos mesmos callbacks jQuery, tornando o código difícil de entender e modificar.

- Performance ruim: por ser um framework legado e que não usa padrões modernos de desenvolvimento em JavaScript, seu código se torna mais lento que a manipulação direta de DOM feita via JS moderno direto.
- Incompatibilidade com práticas modernas: jQuery não se integra bem com bundlers modernos (Webpack, Vite), tipagem estática (TypeScript) ou padrões de acessibilidade atuais.

A arquitetura de backend em J2EE na sua versão legada carrega um conjunto de estruturais que impactam o desenvolvimento e a rápida solução de problemas e manutenção de código.

- Arquitetura monolítica: aplicações J2EE tradicionais tendem a ser grandes monólitos com forte acoplamento entre camadas, dificultando mudanças isoladas e deploys independentes.
- Servidores de aplicação pesados e complicados: exigisse um bom domínio extra de configuração desses servidores que tratam uma especifica J2EE notoriamente verbosa e burocrática.
- Ciclo de desenvolvimento lento: mesmo com poucas alterações no código, exige praticamente um ciclo inteiro de redeploy para se ver o resultado. Recursos como hotdeploy não são tão eficientes e confiáveis nessa estrutura legado.

Esses são alguns dos problemas encontrados na atual arquitetura do SICDT que, individualmente nenhum desses problemas em si são completamente graves, mas o processo com processos lentos, verboses, tecnologias deprecadas e com linguagem antiga, resulta num somatório de problemas tanto para o desenvolvimento, manutenção e usabilidade com o usuário final. O resultado prático é que a infraestrutura e o modelo de arquitetura atual pune a mudança: quanto mais o sistema precisa evoluir, mais caro, arriscado e lento se torna fazê-lo.

Arquitetura desacoplada de microsserviços

A nova arquitetura propõe uma mudança que leva em consideração a correção dos principais problemas que a arquitetura atual acarreta sem mudar a forma como o SICDT em essência trabalha. É importante frisar que o SICDT traz uma nova estrutura de evoluir contratos que torna mais personalizado a configuração via produtos e assinaturas que são poucos os sistemas similares que têm funcionalidades parecidas. Essa característica torna o SICDT mais flexível para parametrizar os produtos para diferentes atender diferentes realidades de clientes e essa característica será mantida.

Portanto, a nova arquitetura trata da parte estrutural e de codificação para tornar o SICDT mais fácil de desenvolver, implantar, testar, com um melhor desempenho e com um estrutura web nova que disponibiliza uma nova “cara” ao sistema condizente com os novos UIX definidos pela CAIXA, tornando essa experiência final melhor para o usuário.

A arquitetura de microsserviços abrange:

- Inicialização rápida: tempos de boots reduzidos.
- Baixo consumo de memória: ideal para containers, sem precisar de um servidor de aplicações como o JBoss por trás.
- Automatização de containers: pode-se instanciar mais de um container microsserviço para gerenciar melhor a carga (via balance loader), dessa forma, distribui melhor a carga se for um serviço muito consumido, e previne melhor contra falhas de infra pois se um serviço cair, o outro poderá continuar ativo.
- Serviços distribuídos por funcionalidade: cada microsserviço pode ser desenvolvido por funcionalidades, o que poderá ser focados em regras de negócio específicas, permitem deploy, escalabilidade e desenvolvimento descentralizados.
- Maior independência de sistemas: um microsserviço pode ser consumido por outros sistemas e por diferentes interfaces de

usuário de um mesmo sistema, o que torna maior a interoperabilidade.

Fig 01. Visão Geral

Os benefícios não apenas em adotar uma arquitetura desacoplada em microsserviços pode trazer resultados mais expressivos, como a escolher de uma framework criado especialmente para isso demonstra uma combinação mais eficiente e prática e, nesse ponto, o Quarkus se encaixa muito bem nessa proposta.

Os benefícios do Quarkus para essa arquitetura são:

Em Produtividade:

- Dev mode com live reload instantâneo: alterações no código são refletidas sem reiniciar o servidor, eliminando o árduo ciclo de reinicialização completa que geralmente um servidor de aplicações demanda.
- Ferramentas quarkus de telemetria e teste: o framework adota diversas ferramentas importantes que ajudam a encontrar falhas, a testar funcionalidades de uma forma rápida e prática.
- Configuração unificada: com apenas um arquivo *application.properties* toda a configuração do Quarkus é estabilidade. Diferente de uma aplicação SICDT na arquitetura atual que abrange arquivos properties, web.xml, jboss.xml, standalone.xml, etc.

Em Desempenho:

- Startup da aplicação rápida: uma aplicação pode ser inicializada de forma completa em poucos segundos.
- Consumo de memória menor e mais eficiente: Quarkus é desenvolvimento para se trabalhar com versões modernas do Java - especialmente 17+, portanto, os benefícios de um pipeline de compilação na GraalVM, execução, gerenciamento de memória e processos JIT que o Java moderno incorpora se torna fundamental para ter um ciclo de desenvolvimento mais rápido e eficiente.

Em Resiliência e Segurança:

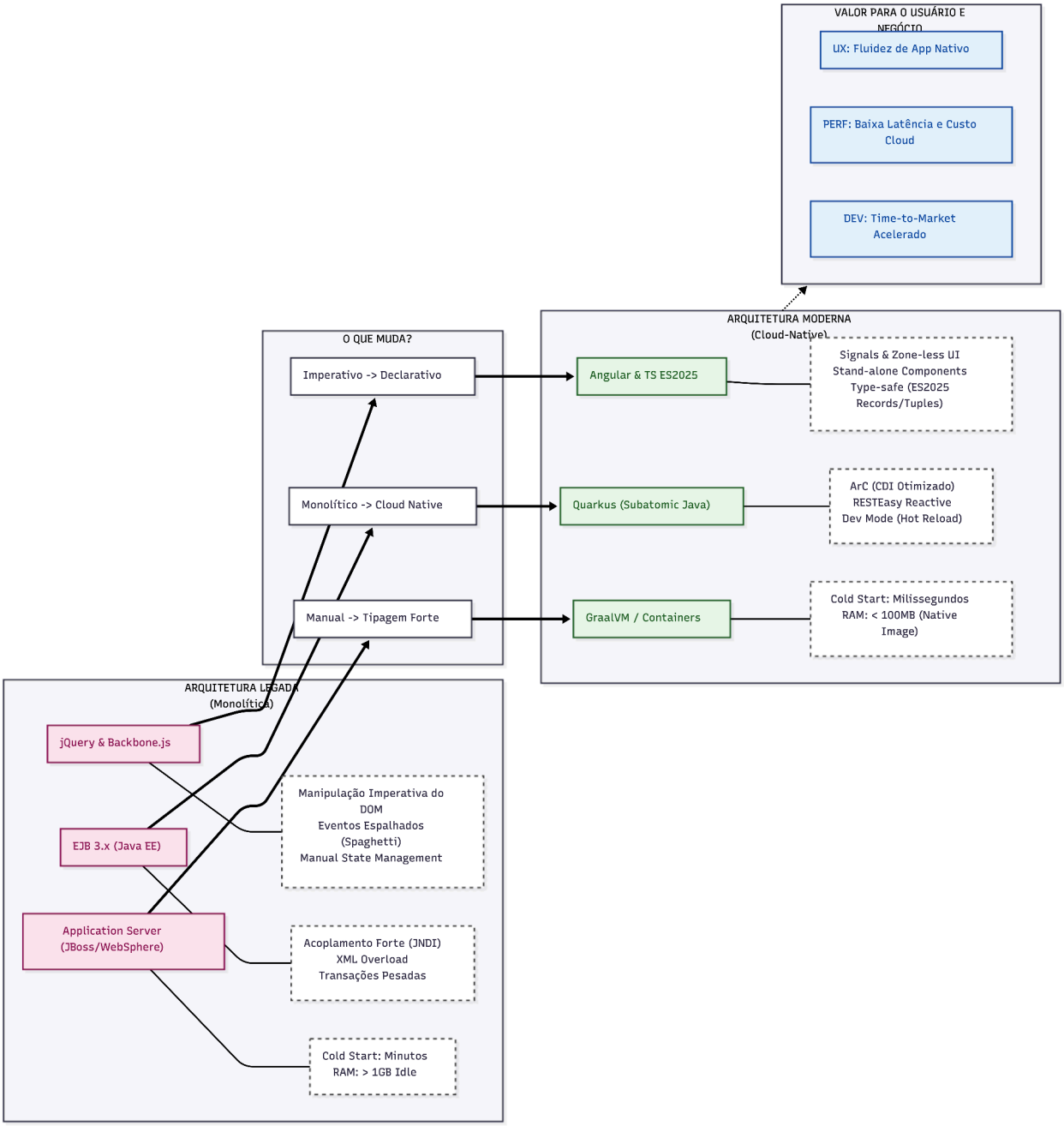
- Integração nativa com SmallRye Fault Tolerance traz circuit breaker, retry, timeout e bulkhead como anotações declarativas diretamente no código de negócio.
- OIDC e JWT validation nativos permitem proteger endpoints com integração direta ao Azure Active Directory sem bibliotecas externas.

- Múltiplas instâncias de um mesmo serviço para gerar redundância contra falhas ou melhorar a performance horizontal controlado por um Load Balancer.

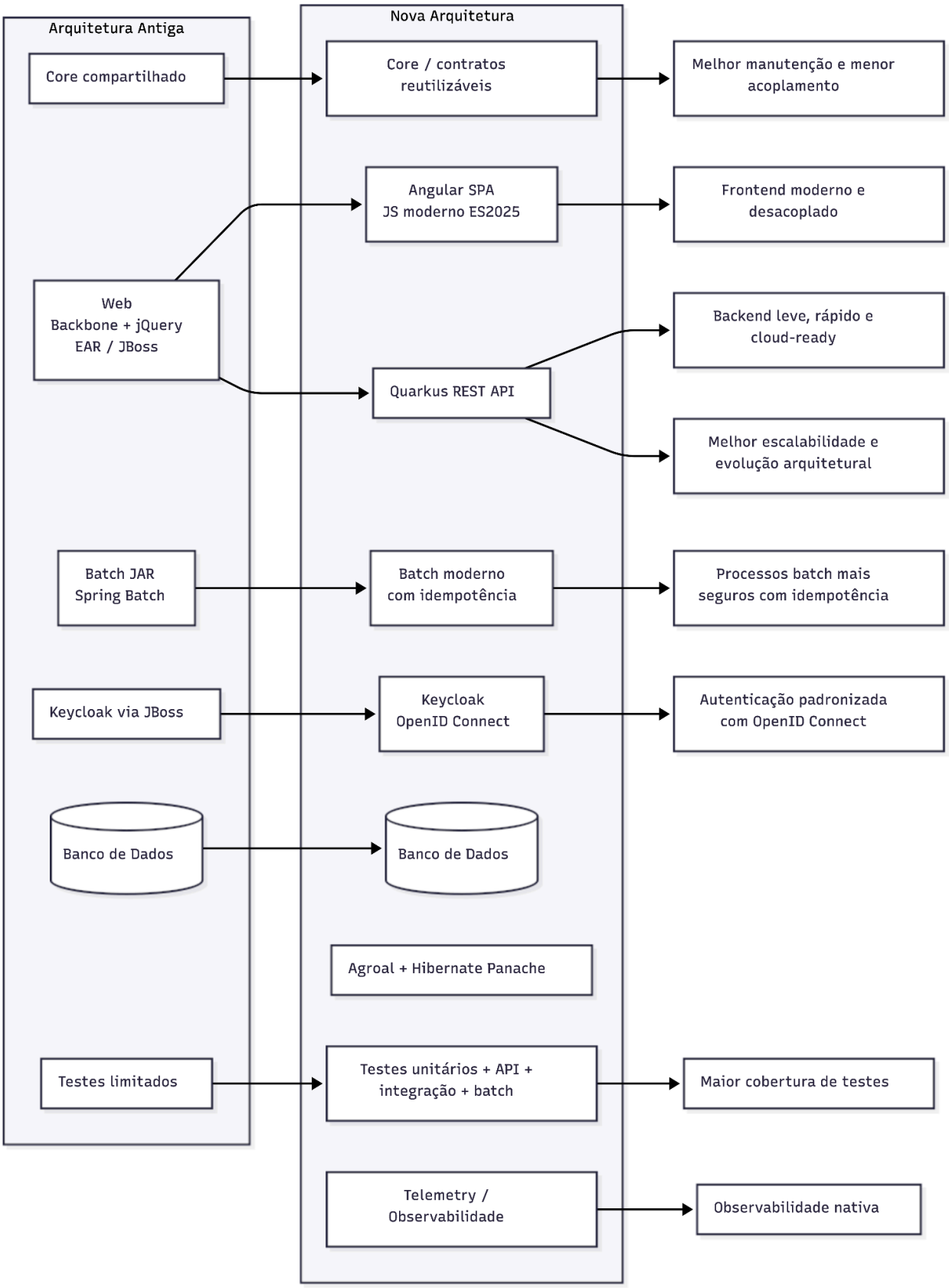
Fig 02. Quarkus

Comparativos de arquitetura

Modelo frontend:



Modelo backend:


Arquitetura Antiga
Core compartilhado
Web
Backbone + jQuery
EAR / JBoss
Batch JAR
Spring Batch
Keycloak via JBoss
Banco de Dados
Testes limitados
Nova Arquitetura
Core / contratos
reutilizáveis
Angular SPA
JS moderno ES2025
Quarkus REST API
Batch moderno
com idempotência
Keycloak
OpenID Connect
Banco de Dados
Agroal + Hibernate Panache
Testes unitários + API +
integração + batch
Telemetry /
Observabilidade
Melhor manutenção e menor
acoplamento
Frontend moderno e
desacoplado
Backend leve, rápido e
cloud-ready
Melhor escalabilidade e
evolução arquitetural
Processos batch mais
seguros com idempotência
Autenticação padronizada
com OpenID Connect
Maior cobertura de testes
Observabilidade nativa

Em resumo, temos:

Os pontos atingidos são onde há problemas no desenvolvimento, no monitoramento, em testes e em performance.

Para um processo antigo de deploy custoso e burocrático, substituímos por um que é muito mais rápido, que consome menos recursos, tem alto desempenho e consegue ser desacoplado.

Para uma interface visual antiga e pesada, substituímos por uma aplicação moderna com visual que possa permitir temas, estruturas mais limpas, rápida e prática para o usuário final.

Para uma cobertura de testes mais amplas que não apenas incorpora testes unitários, mais a camada de testes de integração (que possam permitir regressão